Module 3

Object Oriented Programming

GitHub link: -

https://github.com/Kevin-Maniar/06Aug_Kevin_SE/tree/main/Assignments/Module%203

Index

No	Description	Page no
1	Session – 1	1
2	Session – 2	11
3	Session – 3	22
4	Session – 4	29
5	Session – 5	40
6	Session – 6	46
7	Session – 7	53
8	Mini Project 2	

Session – 1

```c
/*
    Write a simple C program tasklist_c.c that allows you to
    add up to 5 tasks (as strings) to a global array
    and print all tasks using a for loop.
*/

#include<stdio.h>

char tasks [5][100];
int n;

void add_tasks()
{

    printf("How many tasks do you want to add?:-");
    scanf("%d",&n);
    getchar();

    if(n>0 && n<6)
    {
        for(int i=0;i<n;i++)
        {
            printf("Enter Task %d:-",i+1);
            fgets(tasks[i],100,stdin);
        }
    }
    else
    {
        printf("Error You can store only 5 tasks");
    }
}

void print_task()
{
    for(int i=0;i<n;i++)
    {
        printf("Your pending task %d --> %s",i+1,tasks[i]);
    }
}
int main()
{
    add_tasks();
    printf("\n");
```

Session – 1

```
    print_task();
}
```

Output:



Session – 1

```c
/*
    Modify your tasklist_c.c to add a function
    markTaskDone(int index) that sets the
    selected task to 'DONE' in the array,
    then print the updated list.

    Hint:
    Use a separate status array or append ' - DONE' to the task string.
*/

#include<stdio.h>
#include<string.h>

char tasks [5][100];
char status[5][8];
int n;

void add_tasks()
{

    printf("How many tasks do you want to add?:-");
    scanf("%d",&n);
    getchar();

    if(n>0 && n<6)
    {
        for(int i=0;i<n;i++)
        {
            printf("Enter Task %d:-",i+1);
            gets(tasks[i]);
        }
    }
    else
    {
        printf("Error You can store only 5 tasks");
    }
}

void markTaskDone(int index)
{
    strcpy(status[index],"DONE");
}

void print_task()
{
```

Session – 1

```
    for(int i=0;i<n;i++)
    {
        printf("\nTask[%d] --> %s status[%s]",i+1,tasks[i],status[i]);
    }
}
int main()
{
    add_tasks();
    markTaskDone(1);
    print_task();
}
```

Output:



Session – 1

```cpp
/*
    Create a Task class in your favorite OOP language (Java, Python, or C++)
    with properties:
    title and isDone.
    Add a method markDone() that sets isDone to true,
    and a method display() that
    prints the task with its status.
*/

#include<iostream>
using namespace std;

class task
{
    string title = "Pay Gas Bill";
    bool isDone = false;

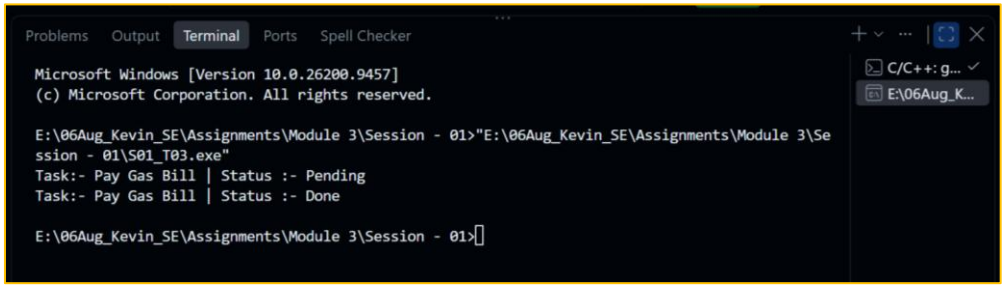
    public:
    void markDone()
    {
        isDone = true;
    }

    void display()
    {
        if(isDone)
        {
            cout <<"Task:- "<<title<<" | Status :- Done"<<endl;
        }
        else
        {
            cout <<"Task:- "<<title<<" | Status :- Pending"<<endl;
        }
    }


};
int main()
{
    task ob;
    ob.display();
    ob.markDone();
    ob.display();
    return 0;
}
```

Session – 1

Output:



Session – 1

```
/*
    Build a simple TaskList class/object that stores multiple Task objects
    and provides addTask(title),
    markTaskDone(index),
    and showTasks() methods.
    Demonstrate adding 3 tasks, marking one as done,
    and displaying all tasks with their statuses.
*/

#include<iostream>
using namespace std;

class task
{
    public:
    string title;
    bool isDone = false;

    void markDone()
    {
        isDone = true;
    }

    void showTask()
    {
        if(isDone)
            cout<<"Task: "<<title<<" | Status:- Done"<<endl;
        else
            cout<<"Task: "<<title<<" | Status: Pending"<<endl;
    }

};

class taskList
{
    task tasks[10];
    int count = 0;

    public:
    void addTask(string T)
    {
        tasks[count].title = T;
        count++;
    }
```

```
    void  markTaskDone(int index)
    {
        tasks[index].markDone();
    }

    void showAll()
    {
        for(int i=0;i<count;i++)
        {
            tasks[i].showTask();
        }
    }
};

int main()
{
    taskList tl;
    tl.addTask("Buy Milk");
    tl.addTask("Do Code");
    tl.addTask("Do workout");

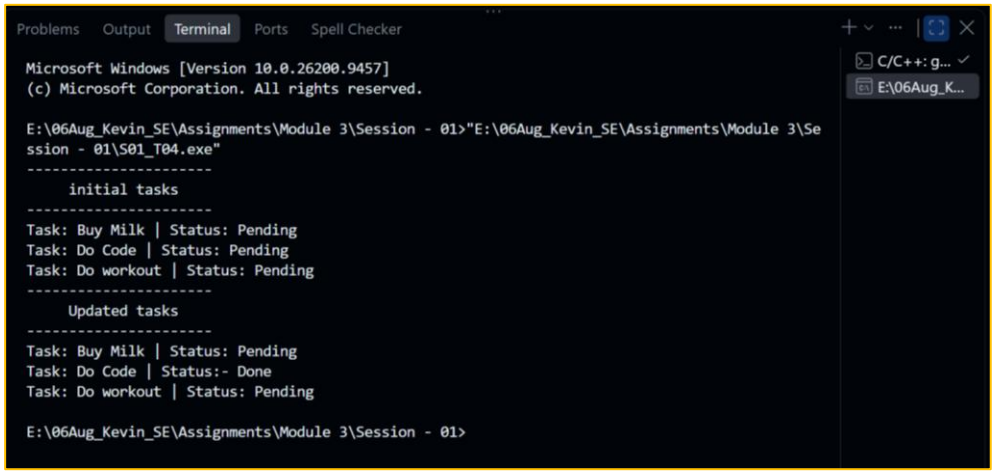
    cout<<"----------------------"<<endl;
    cout<<"     initial tasks  "<<endl;
    cout<<"----------------------"<<endl;

    tl.showAll();
    tl.markTaskDone(1);

    cout<<"----------------------"<<endl;
    cout<<"     Updated tasks  "<<endl;
    cout<<"----------------------"<<endl;
    tl.showAll();
    return 0;
}
```

Session – 1

Output:



Session – 1

```
/*
    Compare your procedural C version and OOP TaskList version:
    List 3 problems you faced in the C version that were solved by using
    classes and methods in the OOP version.


1.) OOP makes it easy to mark task done and pending
2.) OOP support string , with that it's more reliable
3.) In c i used global array which is more efficient if i use class & object */
```

Session – 2

```cpp
/*
    Create a class called Playlist with properties:
    name (string), createdOn (date), and isPublic (boolean).
    Instantiate an object of Playlist and print all its properties.
*/

#include<iostream>
#include<string>
using namespace std;

class PlayList
{
    string name = "S u k o o n";
    string date = "21 - 09 - 2026";
    bool isPublic = true;

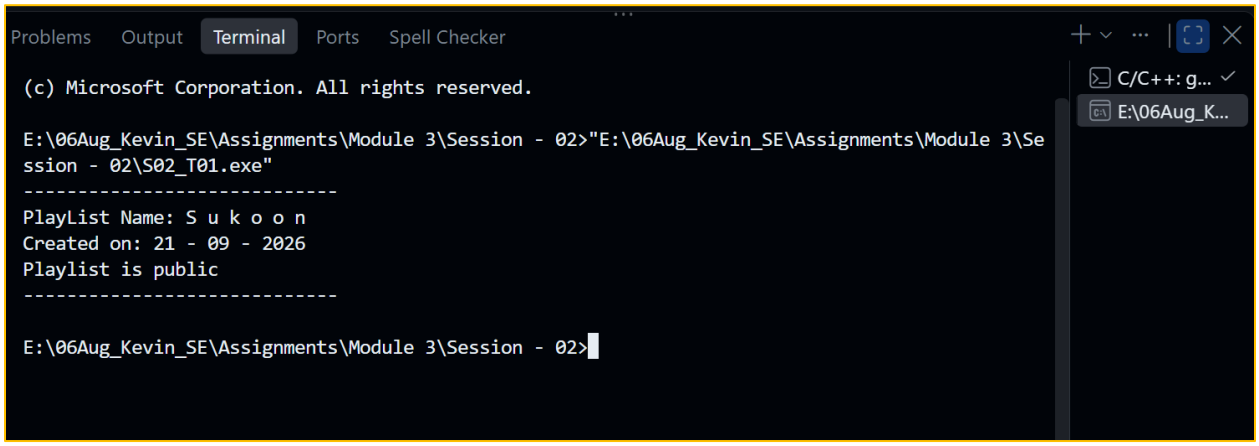
    public:
    void show()
    {
        cout<<"-----------------------------"<<endl;
        cout<<"PlayList Name: "<<name<<endl;
        cout<<"Created on: "<<date<<endl;

        if (isPublic)
            cout<<"Playlist is public"<<endl;
        else
            cout<<"Playlist is private"<<endl;
        cout<<"-----------------------------"<<endl;
    }
};

int main()
{
    PlayList song;
    song.show();
}
```

Session – 2

Output:



Session – 2

```cpp
/*
    Add a member function to your Playlist class
    named togglePublic() that switches
    isPublic between true and false each time it is called.
    Demonstrate by toggling the value twice and printing the result each time.
*/

#include<iostream>
#include<string>
using namespace std;

class PlayList
{
    string name = "S u k o o n";
    string date = "21 - 09 - 2026";
    bool isPublic = false;

    public:
    void show()
    {
        // cout<<"----------------------------"<<endl;
        cout<<"PlayList Name: "<<name<<endl;
        cout<<"Created on: "<<date<<endl;

        if (isPublic)
            cout<<"Playlist is public"<<endl;
        else
            cout<<"Playlist is private"<<endl;
        cout<<"----------------------------"<<endl;
    }

    void togglePublic()
    {
        isPublic = !isPublic;
        // cout<<"Playlist is "<<isPublic;
    }
};

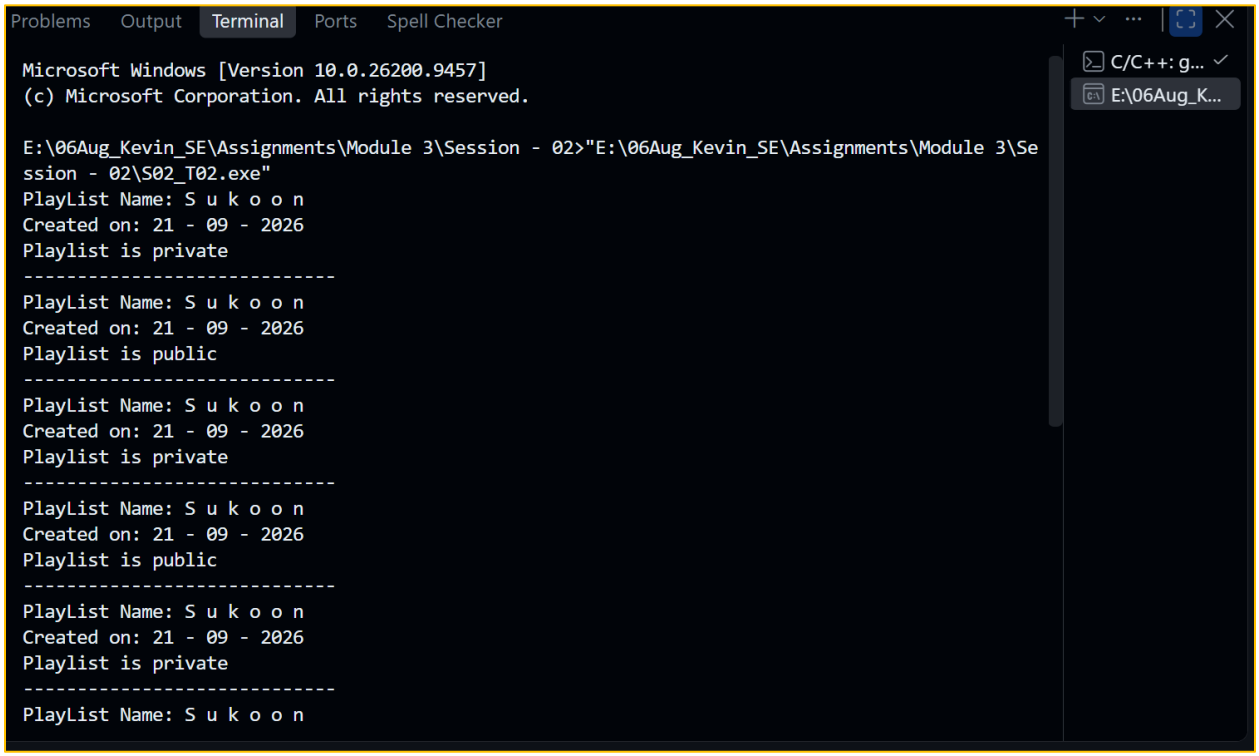
int main()
{
    PlayList song;

    for(int i=0;i<10;i++)
    {
        song.show();
```

Session – 2

```
        song.togglePublic();
    }
}
```

Output::



Session – 2

```
/*
    Create a class called FoodOrder with properties:
    orderId (number),
    restaurantName (string),
    and isDelivered (boolean).
    Write a member function
    markDelivered()
    that sets isDelivered to true and prints a message.
    Instantiate FoodOrder and call markDelivered().
*/

#include<iostream>
using namespace std;

class foodOrder
{
    protected:
    int order_id = 77 ;
    string hotel_name = "Food Bar";
    bool isDelivered = false;

    public:
    void markDelivered()
    {
        isDelivered = true;
        cout<<"Order has been successfully delivered"<<endl;
    }

    void _order()
    {
        cout<<"Order Id: "<<order_id<<endl;
        cout<<"Hotel Name: "<<hotel_name<<endl;
    }

    void _order_status()
    {
        if(isDelivered)
        {
            cout<<"Order is Delivered"<<endl;
        }
        else
        {
            cout<<"Order is Pending"<<endl;
        }
    }
```

Session – 2

```
};

int main()
{
    foodOrder yummy;
    yummy._order();
    yummy._order_status();
    yummy.markDelivered();
    yummy._order_status();
}
```

Output:

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 02>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 02\S02_T03.exe"
Order Id: 77
Hotel Name: Food Bar
Order is Pending
Order has been successfully delivered
Order is Delivered

E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 02>
```

Session – 2

```
/*
    Extend your Playlist class to include a member function
    addSong(songTitle) that adds the song title to
    an array property called songs.

    Demonstrate by adding three song titles and
    displaying the updated songs list.

    Hint:Initialize songs as an empty array inside the constructor.</em>
*/

#include<iostream>
#include<string>
using namespace std;

class PlayList
{
    string name = "S u k o o n";
    string date = "21 - 09 - 2026";
    bool isPublic = true;

    string songs[10];
    int song_count;

    public:

    PlayList()
    {
        song_count = 0;
    }

    void add_song (string Song_title)
    {
        if(song_count < 10)
        {
            songs[song_count] = Song_title;
            song_count++;
        }
    }

    void show()
    {
        cout<<"----------------------------"<<endl;
        cout<<"PlayList Name: "<<name<<endl;
        cout<<"Created on: "<<date<<endl;
```

Session – 2

```
        if (isPublic)
            cout<<"Playlist is public"<<endl;
        else
            cout<<"Playlist is private"<<endl;
        cout<<"----------------------------"<<endl;

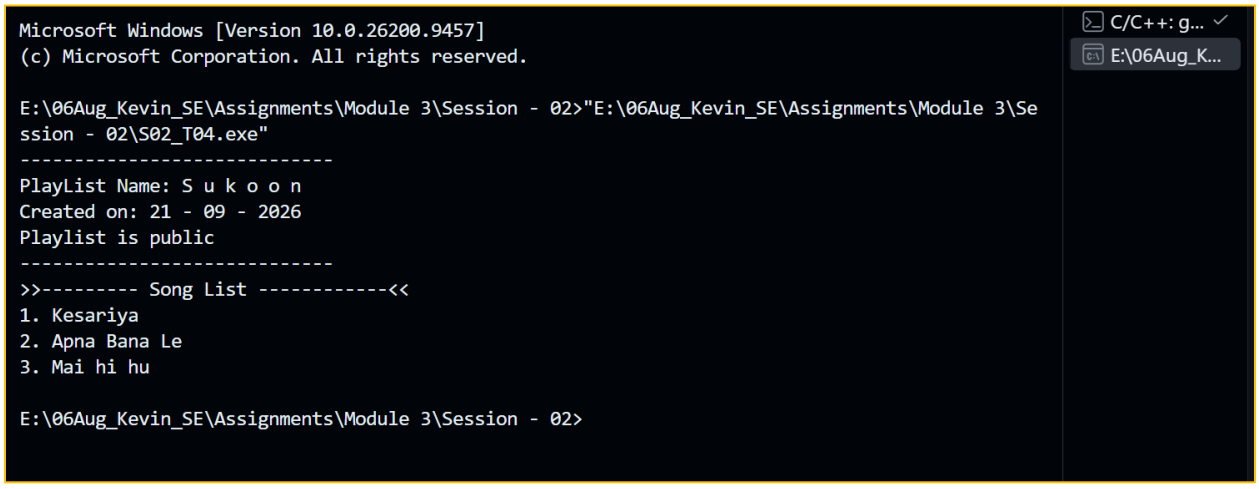
        cout<<">>--------- Song List ------------<<"<<endl;
        if(song_count == 0)
        {
            cout<<"PlayList is empty";
        }
        else
        {
            for(int i=0;i<song_count;i++)
            {
                cout <<i+1<<". "<<songs[i] <<endl;
            }
        }
    }


};

int main()
{
    PlayList ex;
    ex.add_song("Kesariya");
    ex.add_song("Apna Bana Le");
    ex.add_song("Mai hi hu");
    ex.show();
}
```

Session – 2

Output:



Session – 2

```
/*
    Refactor your FoodOrder class so that the constructor takes an object as a
parameter
    (with keys orderId, restaurantName, isDelivered)
    instead of separate arguments.
    Update your instantiation code to use this new constructor signature.
*/

#include<iostream>
using namespace std;


struct Order_data
{
    int o_id;
    string hotel;
    bool isDelivered;
};

class foodOrder
{
    int order_id ;
    string hotel_name ;
    bool isDelivered ;

    public:

    foodOrder(Order_data data)
    {
        order_id = data.o_id;
        hotel_name = data.hotel;
        isDelivered = data.isDelivered;
    }

    void markDelivered()
    {
        isDelivered = true;
        cout<<"Order has been successfully delivered"<<endl;
    }

    void _order()
    {
        cout<<"Order Id: "<<order_id<<endl;
        cout<<"Hotel Name: "<<hotel_name<<endl;
    }
```

Session – 2

```
    void _order_status()
    {
        if(isDelivered)
        {
            cout<<"Order is Delivered"<<endl;
        }
        else
        {
            cout<<"Order is Pending"<<endl;
        }
    }
};

int main()
{
    Order_data myData = {100,"Food Bar",false};

    foodOrder yummy(myData);
    yummy._order();
    yummy._order_status();
    yummy.markDelivered();
    yummy._order_status();
    return 0;
}
```

Output:



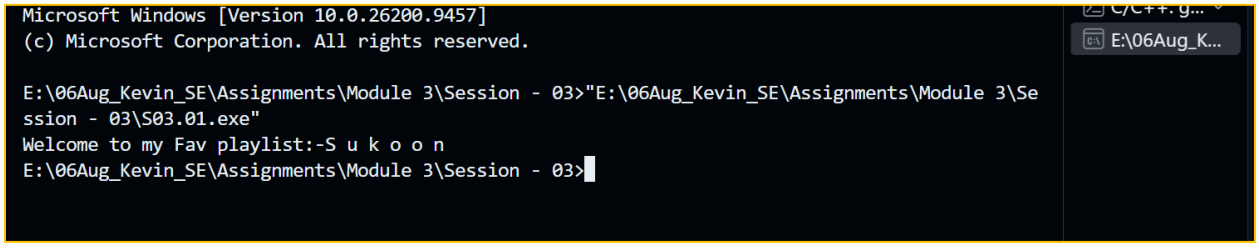
Session – 3

```cpp
/*
    Create a class called Playlist with a default constructor
    that sets the playlist name to 'My Fav'
    and prints a welcome message when an object is created.
*/

#include<iostream>
using namespace std;

class playList
{
    string pl_name;
    public:
    playList()  //Default constructor
    {
        pl_name = "S u k o o n";
        cout<<"Welcome to my Fav playlist:-"<<pl_name;
    }
};
int main()
{
    playList pl;
    return 0;
}
```

Output:



Session – 3

```cpp
/*
    Build a class Product for a Flipkart-style app with
    a parameterized constructor that takes
    productName,
    price, and rating as arguments
    and displays all details using a displayInfo() method.
*/

#include<iostream>
using namespace std;

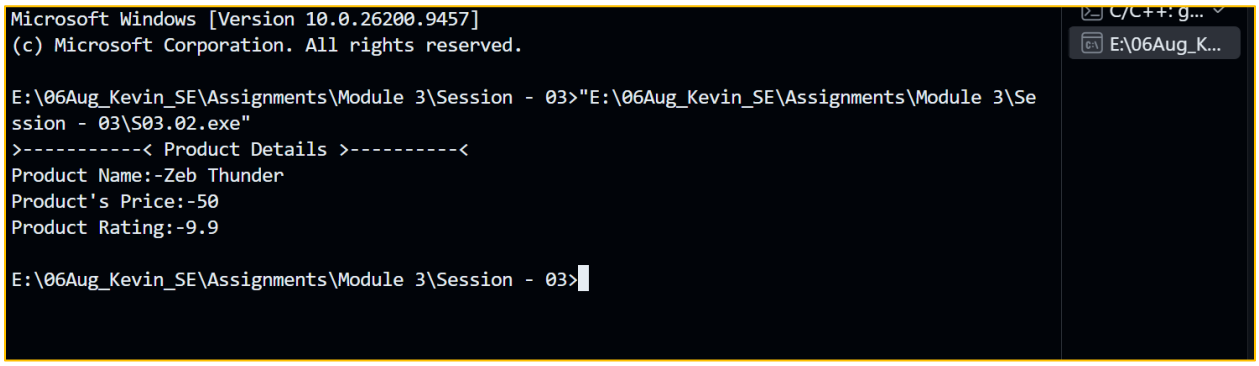
class product
{
    string product_name;
    int product_price;
    double product_rating;

    public:
    product(string _name,int _price,float _rating)
    {
        product_name = _name;
        product_price = _price;
        product_rating = _rating;
    }

     void displayInfo()
        {
            cout<<">-----------< Product Details >----------<"<<endl;
            cout<<"Product Name:-"<<product_name<<endl;
            cout<<"Product's Price:-"<<product_price<<endl;
            cout<<"Product Rating:-"<<product_rating<<endl;
        }
};
int main()
{
    product shop("Zeb Thunder",50,9.9);
    shop.displayInfo();
    return 0;
}
```

Session – 3

Output:



Session – 3

```cpp
/*
    Create a class Movie with a parameterized constructor
    and then use a copy constructor to duplicate a Movie object.
    Print both the original and copied movie
    details to show they are identical
*/

#include<iostream>
using namespace std;

class movie
{
    string _movie;
    float _rating;

    public:
    movie (string title,float rating)
    {
        _movie = title;
        _rating = rating;
    }

    movie(const movie &_copy)
    {
        _movie = _copy._movie;
        _rating = _copy._rating;
    }

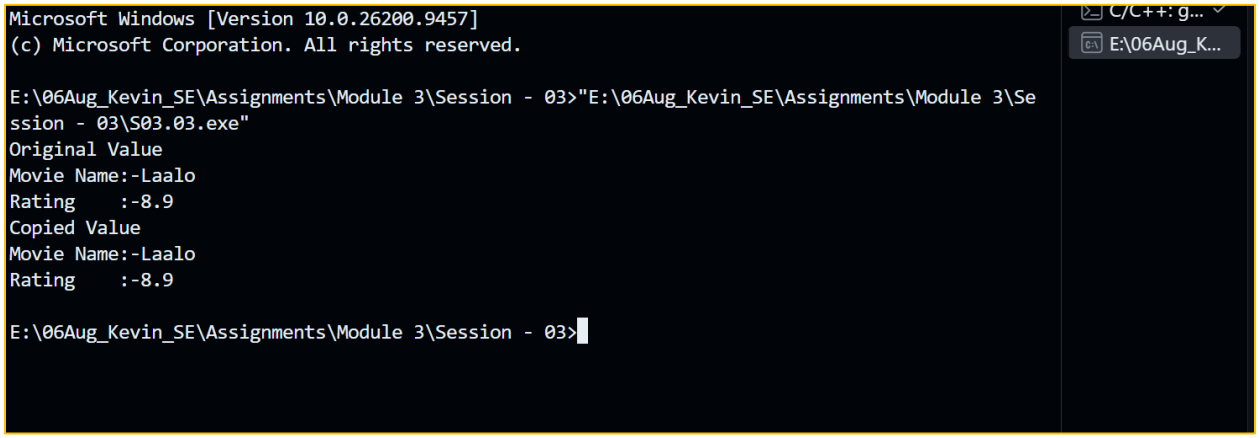
    void showInfo()
    {
        cout<<"Movie Name:-"<<_movie<<endl;
        cout<<"Rating    :-"<<_rating<<endl;
    }
};
int main()
{
    movie obj("Laalo",8.9);

    movie obj2 = obj;

    cout<<"Original Value"<<endl;
    obj.showInfo();
    cout<<"Copied Value"<<endl;
    obj2.showInfo();
    return 0;
```

```
}
```

Output:


Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 03>"E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 03\S03.03.exe"
Original Value
Movie Name:-Laalo
Rating :-8.9
Copied Value
Movie Name:-Laalo
Rating :-8.9
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 03>
C/C++: g...
E:\06Aug_K...

Session – 3

```cpp
/*
    Simulate a BookMyShow ticket booking by creating a Ticket class
    that prints 'Saving your ticket...' in its destructor.
    Create and delete a Ticket object to demonstrate the destructor lifecycle.

    Hint: Use the __del__ method (Python) or ~Ticket() (C++) to define the
destructor.
*/

#include<iostream>
using namespace std;

class bookMyshow
{
    public:
    bookMyshow()
    {
        cout<<"Your ticket is confirmed"<<endl;
    }

    ~bookMyshow()
    {
        cout<<"Saving Your Ticket"<<endl;
    }
};
int main()
{
    bookMyshow ticket;
    return 0;
}
```

Output:

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 03>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 03\S03.04.exe"
Your ticket is confirmed
Saving Your Ticket

E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 03>
```

Session – 3

```
/*
    Modify your Playlist class so that it auto-saves the playlist name to a
    file called autosave.txt when the object is destroyed,
    simulating an auto-save feature like Spotify.
    Hint:
    Write the file-saving code inside the destructor.</em>
*/

#include<iostream>
#include<fstream>
using namespace std;

class playList
{
    private:
    string PL_NAME;

    public:
    playList()
    {
        PL_NAME = "S U K O O N";
        cout<<"My playList:-"<<PL_NAME<<endl;
    }

    ~playList()
    {
        ofstream file("autosave.txt"); // file created
        file<<"My playList Name:-"<<PL_NAME<<endl;
    }
};
int main()
{
    playList pl;
    return 0;
}
```

Output:

```
(c) Microsoft Corporation. All rights reserved.

E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 03>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 03\S03.05.exe"
My playList:-S U K O O N

E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 03>
```

Session – 4

```cpp
/*
    Create a base class called SocialMediaUser with properties username and
followers,
    and a method displayProfile() that prints the username and follower count.
*/

#include<iostream>
using namespace std;

class SocialMediaUSer
{
    protected:
    string _username;
    int _followers;

    public:
    SocialMediaUSer(string _un , int _uf)
    {
        _username = _un;
        _followers = _uf;
    }

    void displayProfile()
    {
        cout<<"Username:-"<<_username<<endl;
        cout<<"Followers:-"<<_followers<<endl;
    }
};
int main()
{
    SocialMediaUSer ig("kevyns.exe",1000);
    ig.displayProfile();
    return 0;
}
```

Session – 4

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 04>"E:\06Aug_Kevin_SE\A
ssion - 04\S04.01.exe"
Username:-kevyns.exe
Followers:-1000

E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 04>
```

Session – 4

```cpp
/*
    Build a YouTuber class that inherits from SocialMediaUser and
    adds a property channelName and a method uploadVideo(title)
    that prints 'Video [title] uploaded to [channelName]'.
*/

#include<iostream>
using namespace std;

class SocialMediaUSer
{
    protected:
    string _username;
    int _followers;

    public:
    SocialMediaUSer(string _un , int _uf)
    {
        _username = _un;
        _followers = _uf;
    }
    void displayProfile()
    {
        cout<<"Username:-"<<_username<<endl;
        cout<<"Followers:-"<<_followers<<endl;
    }
};

class YouTube : public SocialMediaUSer
{
    protected:
    string _channelName;

    public:

    YouTube(string _un, int _uf, string cName) : SocialMediaUSer(_un, _uf)
    {
        _channelName = cName;
    }
    void upload(string vid)
    {
        cout<<"Video '"<<vid<<"' Uploaded to YouTube Channel
"<<_channelName<<endl;
    }
};
```

Session – 4

```
int main()
{
    YouTube yt("Hello.exe",1000,"Any Code");
    yt.displayProfile();
    yt.upload("Code with me");
    return 0;
}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 04>"E:\06Aug_Kevin_SE
ssion - 04\S04.02.exe"
Username:-Hello.exe
Followers:-1000
Video 'Code with me' Uploaded to YouTube Channel Any Code
```

Session – 4

```
/*
    Create a Podcast-er class that also inherits from
    SocialMediaUser and adds a property
    podcastName and a method publishEpisode(episodeTitle)
    that prints 'Episode [episodeTitle] published on [podcastName]'.
*/

#include<iostream>
using namespace std;

class SocialMediaUSer
{
    protected:
    string _username;
    int _followers;

    public:
    SocialMediaUSer(string _un , int _uf)
    {
        _username = _un;
        _followers = _uf;
    }
    void displayProfile()
    {
        cout<<"Username:-"<<_username<<endl;
        cout<<"Followers:-"<<_followers<<endl;
    }
};

class YouTube : public SocialMediaUSer
{
    protected:
    string _channelName;

    public:

    YouTube(string _un, int _uf, string cName) : SocialMediaUSer(_un, _uf)
    {
        _channelName = cName;
    }
    void upload(string vid)
    {
        cout<<"Video '"<<vid<<"' Uploaded to YouTube Channel
"<<_channelName<<endl;
    }
```

```
};

class podCaster : public YouTube
{
    protected:
    string _pdName;

    public:
    podCaster(string _un , int _uf,string cName,string epName) :
YouTube(_un,_uf,cName)
    {
        _pdName = epName;
    }

    void publish(string date)
    {
        cout<<"Episode '"<<_pdName<<"' Published on "<<date<<endl;
    }
};


int main()
{
    podCaster pd("Hello.exe",2500,"AnyIdea","How to master coding?");
    pd.displayProfile();
    pd.publish("09/10/2026");
    pd.upload("Tutorial on Claude");
    return 0;
}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 04>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 04\S04.03.exe"
Username:-Hello.exe
Followers:-2500
Episode 'How to master coding?' Published on 09/10/2026
Video 'Tutorial on Claude' Uploaded to YouTube Channel AnyIdea

E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 04>
```

Session – 4

```
/*
    Demonstrate multilevel inheritance by creating a
    class GamingYouTuber
    that inherits from YouTuber and adds a method
    streamGame(gameName)
    which prints '[username] is now streaming [gameName] on [channelName]'.
*/

#include<iostream>
using namespace std;

class SocialMediaUSer
{
    protected:
    string _username;
    int _followers;

    public:
    SocialMediaUSer(string _un , int _uf)
    {
        _username = _un;
        _followers = _uf;
    }
    void displayProfile()
    {
        cout<<"Username:-"<<_username<<endl;
        cout<<"Followers:-"<<_followers<<endl;
    }
};

class YouTube : public SocialMediaUSer
{
    protected:
    string _channelName;

    public:

    YouTube(string _un, int _uf, string cName) : SocialMediaUSer(_un, _uf)
    {
        _channelName = cName;
    }
    void upload(string vid)
    {
        cout<<"Video '"<<vid<<"' Uploaded to YouTube Channel
"<<_channelName<<endl;
```

```
    }
};

class gaming_YouTuber : public YouTube
{

    public:
    gaming_YouTuber(string _un, int _uf, string cName) : YouTube(_un,_uf,cName)
    {
        //
    }
    void stream_game(string game_name)
    {
        cout<<_username<<" is now streaming "<<game_name<<" on
"<<_channelName<<"Youtube Channel"<<endl;
    }
};
int main()
{
    gaming_YouTuber ob("Hello.exe",2500,"AnyIdea");
    ob.stream_game("Chess");
    return 0;
}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 04>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 04\S04.04.exe"
Hello.exe is now streaming Chess on AnyIdeaYoutube Channel
```

Session – 4

```
/*
    Implement hierarchical inheritance by creating an
    InstagramInfluencer class that inherits from SocialMediaUser
    and adds a method postStory(storyTitle)
    which prints '[username] posted a new story: [storyTitle]'.

    Hint: Think about how SocialMediaUser is the parent for YouTuber, Podcast-er,
and InstagramInfluencer.
*/

#include<iostream>
using namespace std;

class SocialMediaUSer
{
    protected:
    string _username;
    int _followers;

    public:
    SocialMediaUSer(string _un , int _uf)
    {
        _username = _un;
        _followers = _uf;
    }
    void displayProfile()
    {
        cout<<"Username:-"<<_username<<endl;
        cout<<"Followers:-"<<_followers<<endl;
    }
};

class YouTube : public SocialMediaUSer
{
    protected:
    string _channelName;

    public:

    YouTube(string _un, int _uf, string cName) : SocialMediaUSer(_un, _uf)
    {
        _channelName = cName;
    }
    void upload(string vid)
    {
```

```
        cout<<"Video '"<<vid<<"' Uploaded to YouTube Channel
"<<_channelName<<endl;
    }
};

class podCaster : public SocialMediaUSer
{
    protected:
    string _pdName;

    public:
    podCaster(string _un , int _uf) : SocialMediaUSer(_un,_uf)
    {
        // _pdName = epName;
    }

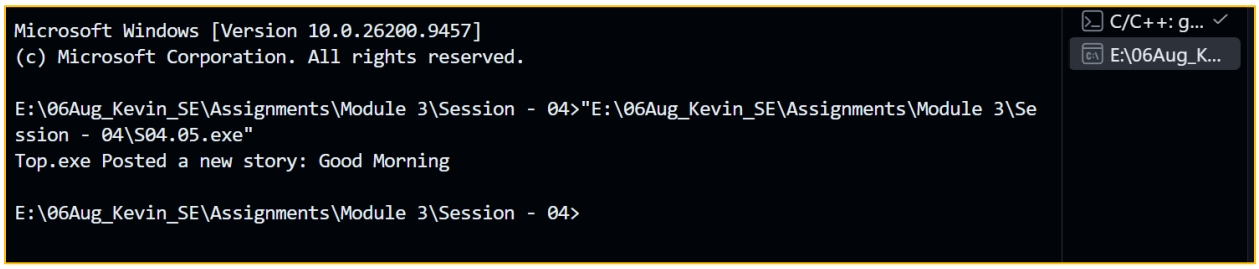
    void publish(string date)
    {
        cout<<"Episode '"<<_pdName<<"' Published on "<<date<<endl;
    }
};

class inst_influencer : public SocialMediaUSer
{
    public:
    inst_influencer(string _un , int _uf) : SocialMediaUSer(_un,_uf)
    {

    }
    void post_story(string _story)
    {
        cout<<_username<<" Posted a new story: "<<_story<<endl;
        //which prints '[username] posted a new story: [storyTitle]'.
    }
};
int main()
{
    inst_influencer ig("Top.exe",34);
    ig.post_story("Good Morning");
    return 0;
}
```

Session – 4

Output:



Session – 5

```cpp
/*
    Create a Java class called PaymentProcessor with
    two overloaded methods processPayment():
    one that takes only an amount,
    and one that takes amount and a coupon code.
    Print which version is called and the final amount in each case.
*/

#include<iostream>
using namespace std;

class paymentProcessor
{
    public:
    void process_Payment(int amt)
    {
        cout << "Amount is :-" << amt << endl;
        cout << "Version - 1 executed Successfully" << endl;
    }

    void process_Payment(int amt,int _cup_code)
    {
        cout <<"Amount:-" << amt << endl
        <<"coupon:-"<<_cup_code<<endl;
        cout << "Version - 2 executed Successfully" <<endl;
    }
};
int main()
{
    paymentProcessor ob;
    ob.process_Payment(50);
    cout << "-------------------" << endl;
    ob.process_Payment(10,23659283);
    return 0;
}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 05>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 05\S05_T01.exe"
Amount is :-50
Version - 1 executed Successfully
-------------------
Amount:-10
coupon:-23659283
Version - 2 executed Successfully

E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 05>
```

Session – 5

```
/*
    Build two classes,
    InstagramUploader and YouTubeUploader,
    each with a method uploadContent().
    Both should extend a base class SocialMediaUploader and override
uploadContent()
    to print a message showing how uploading works differently for Instagram and
YouTube.
*/

#include<iostream>
using namespace std;

class SocialMediaUploader
{
    public:
    void uploadContent()
    {
        cout<<"Uploading Content on generic social media platform";
    }
};

class instagramUploader : public SocialMediaUploader
{
    public:
    void uploadContent()
    {
        cout<<"Instagram Creator"<<endl;
    }
};

class YouTubeUploader : public SocialMediaUploader
{
    public:
    void uploadContent()
    {
        cout<<"YouTube Creator"<<endl;
    }
};

int main()
{
    instagramUploader ig;
    YouTubeUploader yt;
    ig.uploadContent();
```

```
    yt.uploadContent();

    return 0;
}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 05>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 05\S05_T02.exe"
Instagram Creator
YouTube Creator
```

Session – 5

```
/*
    Write a function in Java or Python that simulates a Flipkart-style search:
overload a method searchProduct()
    to allow searching by product name or by product name and category.
Demonstrate both usages with sample data.
*/

#include<iostream>
using namespace std;
class overload_1
{
    public:
    void SEARCH_PRODUCT(string product)
    {
        cout<<" | Search Mode | Product "<<endl;
        cout<<" Searching Product:- "<<product<<endl;

    }

    void SEARCH_PRODUCT(string product , string categories)
    {
        cout<<" | Search Mode | Product + Category "<<endl;
        cout<<" Searching Product:- "<<product <<" in category
"<<categories<<endl;
    }


};

int main()
{
    overload_1 ob;
    ob.SEARCH_PRODUCT("iphone");
    ob.SEARCH_PRODUCT("iphone","Mobile");
    return 0;
}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 05>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 05\S05_T03.exe"
 | Search Mode | Product
 Searching Product:- iphone
 | Search Mode | Product + Category
 Searching Product:- iphone in category Mobile
```

Session – 5

```
/*
    Given this code:

class MusicPlayer
{
    void play(String song)
    {
        System.out.println("Playing: " + song);
    }
}

class SpotifyPlayer extends MusicPlayer
{
    void play(String song)
    {
        System.out.println("Streaming on Spotify: " + song);
    }
}
    Create an object of type MusicPlayer but assign it a SpotifyPlayer instance,
then call play().

    Explain the output.
    Hint:
    This tests runtime polymorphism (overriding) and dynamic method
dispatch.</em>
 */

#include<iostream>
using namespace std;
class MusicPlayer
{   public:
    virtual void play(string song)
    {
        cout<<"Playing:-"<<song<<endl;
    }
};

class SpotifyPlayer : public MusicPlayer
{
    public:
    void play(string song) override
    {
        cout<<"Streaming on Spotify:-"<<song<<endl;
    }
};
```

Session – 5

```
int main()
{
    MusicPlayer* ob = new SpotifyPlayer;
    ob->play("Sunn");
    delete ob;
    return 0;
}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 05>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 05\S05_T04.exe"
Streaming on Spotify:-Sunn
```

Session – 6

```cpp
/*
    Create a class called Song in your preferred OOP language with private properties title and artist.
    Add public getter and setter methods to access and modify these properties, then create an object and update its title.
*/

#include<iostream>
using namespace std;

class Song
{
    private:
    string title,artist;

    public:
    void getter()
    {
        title = "Spiderman";
        artist = "Tom Holand";

        cout<<"Getter() Called"<<endl;
        cout<<"Title  : "<<title<<endl;
        cout<<"Artist : "<<artist<<endl;
    }

    void setter()
    {
        title = "Krish";
        // artist = "Hrithik Roshan";

        cout<<"Setter() Called"<<endl;
        cout<<"Title  : "<<title<<endl;
        cout<<"Artist : "<<artist<<endl;
    }
};
int main()
{
    Song s;
    s.getter();
    s.setter();
    return 0;
}
```

Session – 6

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 06>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 06\S06_T01.exe"
Getter() Called
Title  : Spiderman
Artist : Tom Holand
Setter() Called
Title  : Krish
Artist : Tom Holand
```

Session – 6

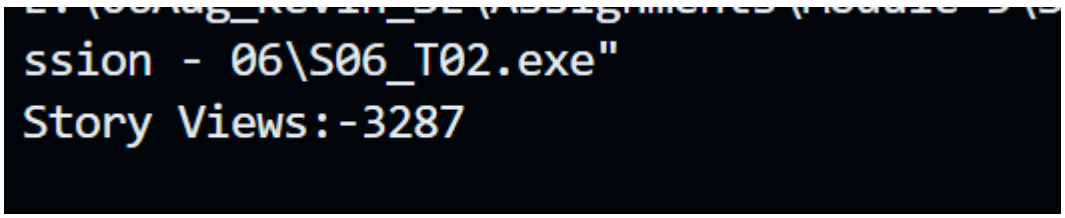
```cpp
/*
Build a class called Instagram_Story with a protected property storyViews.
Create a subclass called SponsoredStory that can access and display the
storyViews value.
*/

#include<iostream>
using namespace std;

class Instagram_story
{
    protected:
    int STORY_VIEWS;
};

class Sponsored_story : protected Instagram_story
{
    public:
    void access()
    {
        STORY_VIEWS = 3287;
        cout<<"Story Views:-"<<STORY_VIEWS<<endl;
    }
};
int main()
{
    Sponsored_story ob;
    ob.access();
    return 0;
}
```

Output:



Session – 6

```cpp
/*
    Imagine a Flipkart-like app:
    create an abstract class Product with an abstract method upload().
    Then, create two subclasses,
    Electronics and Clothing,
    that each implement the upload() method to print a different upload message.
*/

#include<iostream>
using namespace std;

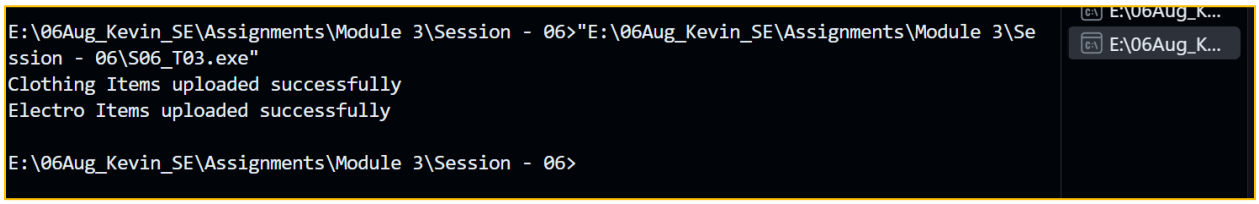
class product
{
    public:
    virtual void upload()= 0;
};

class electro : public product
{
    public:
    void upload () override
    {
        cout<<"Electro Items uploaded successfully"<<endl;
    }
};

class cloth : public product
{
    public:
    void upload() override
    {
        cout<<"Clothing Items uploaded successfully"<<endl;
    }
};
int main()
{
    cloth c;
    c.upload();
    electro e;
    e.upload();
    return 0;
}
```

Session – 6

Output:



Session – 6

```
/*  Refactor the following code so that the
    user's phone number in the UserProfile class is private and
    can only be set or retrieved using public methods
    Hint:Add private access modifier to the phone number and
        create setPhoneNumber() and getPhoneNumber() methods.
*/

#include<iostream>
using namespace std;

class UserProfile
{
    private:
    long long phone;
    public:
    void setPhoneNumber()
    {
        phone = 8050480504;
    }

    void getPhoneNumber()
    {
        cout<<"phone : "<<phone;
    }
};

int main()
{
  UserProfile up;
  up.setPhoneNumber();
  up.getPhoneNumber();

}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session - 06>"E:\06Aug_Kevin_SE\Assignments\Module 3\Se
ssion - 06\S06_T04.exe"
phone : 8050480504
```

Session – 6

```
/*
    Explain in your own words the difference between encapsulation and
abstraction,
    using examples from any social media app you use daily (like Instagram or
WhatsApp).
*/

// ANSWER:
/* 1. Abstraction (Hiding Complexity)
    What it means: Abstraction focuses on what an object does, not how it does
it. It hides complex internal mechanics and presents a simple, clean interface to
the user.

    WhatsApp Example: When you send a voice note on WhatsApp, you just tap and
hold the microphone icon. Under the hood, the app is handling audio recording,
data compression, packet encryption, network routing, and server delivery.
However, you don't need to know any of that complexity—you are given a simple
button to interact with. That is abstraction.

2. Encapsulation (Protecting Data)
    What it means: Encapsulation is the practice of bundling data and methods
together into a single unit (like a class) and restricting direct access to the
internal data from the outside world (data hiding).

    Instagram Example: Think of your Instagram account profile. Your sensitive
data—like your password, email, and phone number—are kept private inside the
account system. Other users or external apps cannot just walk up and change your
password directly (account.password = "hacked"). Instead, Instagram wraps that
data inside a secure class and provides controlled public methods, like a
changePassword() function, which safely checks your old password before allowing
changes.

*/
```

Session – 7

```cpp
/*
    Create a text file called my_fav_songs.txt and write the names of your 5 favorite songs into it using ofstream.
*/

#include<iostream>
#include<fstream>
using namespace std;

int main()
{
    ofstream ob("my_fav_songs.txt",ios::app);
    cout<<"File is Created"<<endl;

    ob<<"Below is My favorite songs"<<endl;
    ob<<"----------------------------"<<endl;

    ob<<"1. “Like a Rolling Stone” by Bob Dylan (1965)"<<endl;
    ob<<"2. “Respect” by Aretha Franklin (1967)"<<endl;
    ob<<"3. “Billie Jean” by Michael Jackson (1982)"<<endl;

    cout<<"Data inserted in file successfully"<<endl;
}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session 07>"E:\06Aug_Kevin_SE\Assignments\Module 3\Session 07\S07_T01.exe"
File is Created
Data inserted in file successfully

E:\06Aug_Kevin_SE\Assignments\Module 3\Session 07>
```

Session – 7

```cpp
/*
    Read all song names from my_fav_songs.txt using ifstream and display each
song on a new line in the console.
*/

#include<iostream>
#include<fstream>
using namespace std;

int main()
{
    ifstream ob("my_fav_songs.txt");
    string str;

    while(getline(ob,str))
    {
        cout<<str<<endl;
    }

}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session 07>"E:\06Aug_Kevin_SE\Assignments\Module 3\Sess
ion 07\S07_T02.exe"
Below is My favorite songs
---------------------------
1. Γ£Like a Rolling StoneΓ¥ by Bob Dylan (1965)
2. Γ£RespectΓ¥ by Aretha Franklin (1967)
3. Γ£Billie JeanΓ¥ by Michael Jackson (1982)
 Lag ja gale
Below is My favorite songs
---------------------------
1. Γ£Like a Rolling StoneΓ¥ by Bob Dylan (1965)
2. Γ£RespectΓ¥ by Aretha Franklin (1967)
3. Γ£Billie JeanΓ¥ by Michael Jackson (1982)

E:\06Aug_Kevin_SE\Assignments\Module 3\Session 07>
```

Session – 7

```
/*
    Modify your code to append a new song name entered by the user to
my_fav_songs.txt without overwriting the existing list.
    Hint:Open the file in append mode using ofstream.</em>
*/

#include<iostream>
#include<fstream>
using namespace std;

int main()
{
    ofstream ob("my_fav_songs.txt",ios::app);
    cout<<"File is Created"<<endl;

    string fav_song;

    cout<<"Dear! What's your fav song?";
    getline(cin,fav_song);

    ob<<fav_song<<endl;
    cout<<"Data inserted in file successfully"<<endl;

    return 0;
}
```

Output:

```
E:\06Aug_Kevin_SE\Assignments\Module 3\Session 07>"E:\06Aug_Kevin_SE\Assignments\Module 3\Sess
ion 07\S07_T03.exe"
File is Created
Dear! What's your fav song?Sunn
Data inserted in file successfully

E:\06Aug_Kevin_SE\Assignments\Module 3\Session 07>
```

Session – 7

```cpp
/*
    Build a Flipkart-style wishlist tracker: ask the user to enter 3 product names and prices,
    save them to a file called wishlist.txt, then read the file and display each product with its price.
*/

#include<iostream>
#include<fstream>
using namespace std;

class wishlist
{
    public:
    string product_name;
    int product_price;
};

class tracker : public wishlist
{
    wishlist wish[10];
    // int count = 0;
    public:

    void take_data()
    {
        for(int i=0;i<3;i++)
        {
            cout<<"Enter product "<<(i+1)<<" name: ";
            getline(cin, wish[i].product_name);

            cout<<"Enter product "<<(i+1)<<" price: ";
            cin>>wish[i].product_price;
            getchar();
        }
    }

    void write_data()
    {
        ofstream file("Wishlist.txt");
        cout<<"file is created"<<endl;

        for(int i=0;i<3;i++)
        {
            file<<"Product "<<(i+1)<<"'s Name: "<<wish[i].product_name<<endl;
```

Session – 7

```
            file<<"Product "<<(i+1)<<"'s Price: "<<wish[i].product_price<<endl;
            file<<"-----------------------"<<endl;
        }
    }

    void read_data()
    {
        ifstream file("wishlist.txt");

        string str;

        while(getline(file,str))
        {
            cout<<str<<endl;
        }
    }
};

int main()
{
       tracker t;
       t.take_data();
       t.write_data();
       t.read_data();
}
```

Output:

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

E:\06Aug_Kevin_SE\Assignments\Module 3\Session 07>"E:\06Aug_Kevin_SE\Assignments\Module 3\Sess
ion 07\S07_T04.exe"
Enter product 1 name: Keyboard
Enter product 1 price: 150
Enter product 2 name: Mouse
Enter product 2 price: 100
Enter product 3 name: CPU
Enter product 3 price: 5000
file is created
Product 1's Name: Keyboard
Product 1's Price: 150
-----------------------
Product 2's Name: Mouse
Product 2's Price: 100
-----------------------
Product 3's Name: CPU
Product 3's Price: 5000
-----------------------

E:\06Aug_Kevin_SE\Assignments\Module 3\Session 07>
```

Session – 7